

What is an Operating System?

An **Operating System (OS)** is **system software** that manages computer **hardware and software resources** and acts as an **interface between the user and the computer hardware**.

Functions of an Operating System

(Also called **services provided by the OS**)

1. Process Management

- Manages all programs (processes) running on the computer.
- Decides **which process gets CPU time** and for how long.
- Handles **process creation, scheduling, and termination**.

2. Memory Management

- Controls the **main memory (RAM)**.
- Allocates memory to programs when they start.
- Frees memory when programs finish execution.
- Prevents memory conflicts between programs.

3. File Management

- Manages **files and directories** on storage devices.
- Allows users to **create, read, write, and delete files**.
- Maintains file organization and access permissions.

4. Device Management

- Controls and coordinates **input and output devices**.
- Uses **device drivers** to communicate with hardware.
- Ensures devices like keyboard, mouse, printer, and monitor work properly.

5. User Interface

- Provides a way for users to interact with the computer.
- Types of interfaces:
 - **Graphical User Interface (GUI)** – icons, windows, menus
 - **Command Line Interface (CLI)** – text commands

6. Security and Protection

- Protects system data from **unauthorized access**.
- Uses **user authentication**, passwords, and access control.
- Prevents misuse of system resources.

Types of Operating Systems

1. Batch Operating System

- A **Batch Operating System** is an operating system in which **similar jobs are collected and executed in batches**, with **no user interaction during execution**.

How It Works

1. Users submit jobs (programs with required data).
2. The operating system stores jobs in a **queue**.
3. Jobs with similar requirements are grouped into batches.
4. The OS executes jobs **one by one**.
5. Output is produced **after all jobs are completed**.

Advantages

- Efficient for large and repetitive jobs
- Better CPU utilization
- Suitable for bulk data processing

Disadvantages

- Long waiting time
- Difficult to detect and correct errors
- Not suitable for interactive applications

2. Multiprogramming Operating System

- A **Multiprogramming Operating System** allows **multiple programs to reside in main memory at the same time** so that the CPU is never idle.

How It Works

1. Several programs are loaded into main memory.
2. One program uses the CPU.
3. When it waits for I/O, the CPU switches to another program.

4. This continues until all programs complete.

Advantages

- High CPU utilization
- Increased system throughput
- Less idle CPU time compared to batch OS

Disadvantages

- Complex memory management
- No guarantee of quick response
- Requires efficient CPU scheduling

3. Multitasking Operating System

- A **Multitasking Operating System** allows a **single user** to run **multiple applications at the same time**.

How It Works

1. CPU time is divided into **small time slices**.
2. Each task gets CPU time in turn.
3. The OS switches rapidly between tasks.

Advantages

- Improves user productivity
- Smooth and responsive system
- Multiple applications can run together

Disadvantages

- Performance may reduce with many tasks
- Requires more memory and CPU power

4. Time Sharing Operating System

- A **Time Sharing Operating System** allows **multiple users** to use a computer system **simultaneously** by sharing CPU time.

How It Works

1. Multiple users are connected to the system.
2. CPU time is divided into small time slices.
3. Each user gets a time slice.
4. The CPU switches rapidly between users.

Advantages

- Efficient use of system resources

- Supports multiple users at the same time

Disadvantages

- Performance decreases with many users
- Security and data privacy issues
- Requires effective scheduling

5. Real Time Operating System (RTOS)

- A **Real Time Operating System** is an operating system that **produces output within a fixed and guaranteed time limit**.
- The system is correct **only if the result is produced within the deadline**.

How It Works

1. An event or input occurs.
2. RTOS processes the task immediately.
3. The task must finish before its deadline.
4. Missing the deadline may cause system failure.

Types of RTOS

1. Hard Real Time Operating System

- Deadlines **must never be missed**
- Missing a deadline causes system failure

Examples

- Airbag control systems
- Medical life-support systems

2. Soft Real Time Operating System

- Deadlines are important but **not strict**
- Occasional delays are acceptable

Examples

- Multimedia systems
- Online streaming applications

Advantages of RTOS

- Very fast response
- High reliability
- Suitable for critical applications

Disadvantages of RTOS

- Expensive to develop
- Limited features
- Complex system design

What is a System Call?

- A **system call** is a **request made by a user program to the operating system** to perform a task that requires access to system resources.
- A user program **cannot directly access hardware** such as CPU, memory, disk, or printer.
- When it needs such access, it uses a **system call**, and the operating system performs the task **safely and securely**.

Uses of System Calls

System calls are used to:

- Access hardware resources in a safe manner
- Perform file operations like **open, read, write, close**
- Create, manage, and terminate processes
- Communicate with the operating system

Why System Calls are Needed

- Protects system resources from misuse
- Ensures security and stability
- Separates user mode from kernel mode

1. fork()

- Used to **create a new process**
- The newly created process is called the **child process**
- The child process is a **copy of the parent process**
- Both processes run independently

2. exec()

- Used to **replace the current process** with a new program
- The process ID remains the same

3. read()

- Used to **read data** from a file or input device
- Transfers data from device to program memory

4. write()

- Used to **write data** to a file or output device
- Transfers data from program memory to device

Steps Performed by OS to Serve a System Call

When a program makes a system call, the operating system performs the following steps:

1. **System Call Request**

The user program requests a service by calling a system call function (for example, `read()`, `write()`, `fork()`).

2. **Trap to Kernel Mode**

The CPU switches from **user mode** to **kernel mode** using a special instruction called a trap or interrupt. This ensures secure access to system resources.

3. **Identify the System Call**

The OS checks the system call number provided by the program to determine which service is requested.

4. **Execute the Service**

The kernel performs the requested operation, such as reading a file, creating a process, or allocating memory.

5. **Return Result to User Program**

After completing the task, the OS switches back to user mode and returns the result (success, failure, or data) to the program.

Example of a System Call: `read()`

Suppose a program wants to read data from a file. The following steps occur:

1. **System Call Invocation**

The program calls the system call:

`read(fd, buffer, size)`

- `fd` = file descriptor
- `buffer` = memory area to store data
- `size` = number of bytes to read

2. **Switch to Kernel Mode**

The CPU transfers control from user mode to kernel mode using a trap instruction.

3. **Validation by OS**

The operating system checks whether the file descriptor is valid and whether the process has permission to read the file.

4. **Perform the Operation**

The OS reads the requested data from the disk into a kernel buffer and then copies it to the user buffer.

5. **Return to User Mode**

The OS returns the number of bytes read (or an error code) and switches control back to user mode.

Four Events That Cause Process Creation

1. **System Initialization (Boot Time)**

When the computer starts, the operating system creates essential system processes needed to run basic services.

2. **Process Creation by Another Process**

A running process can create a new process using system calls (like fork). The new one is called a child process.

3. **User Request to Start a Program**

When a user opens an application or runs a command, the OS creates a new process to execute that program.

4. **Submission of a Batch Job**

In batch systems, when a job is submitted, the operating system creates a process to handle and execute that job.

Layered Structure of an Operating System

The **layered structure** divides an operating system into **separate layers**, where **each layer has a specific function**.

Each layer communicates **only with the layer directly above or below it**.

This approach makes the operating system:

- Easier to design

- Easier to implement
- Easier to test and debug

Typical Layered Structure of an Operating System

(From bottom to top)

1. Hardware Layer

- Includes physical components such as **CPU, main memory, disks, and input/output devices**.
- Provides the basic resources required by the operating system.
- Forms the **foundation** of all other layers.

2. Kernel Layer

- Core part of the operating system.
- Manages **CPU scheduling, memory management, and device control**.
- Handles **process management, file management, and resource allocation**.

3. System Call Layer

- Provides an interface between **user programs and the kernel**.
- User programs request services through **system calls**.
- Ensures safe access to hardware and system resources.

4. User Interface Layer

- Allows users to interact with the operating system.
- Can be:
 - **Command Line Interface (CLI)** like a shell
 - **Graphical User Interface (GUI)** with windows and icons
- Translates user commands into system calls.

5. Application Layer

- Contains user applications such as **web browsers, text editors, and games**.
 - Applications use operating system services to perform tasks.
 - Topmost layer in the structure.
-

Kernel

The **kernel** is the **core part of an operating system**.

It manages the system's **hardware and resources** and lets software talk to the hardware.

Microkernel

A **microkernel** is a type of kernel that keeps **only the most essential functions** inside the kernel.

It handles:

- CPU scheduling
 - Memory management
 - Inter-process communication (IPC)
-

Exokernel

An **exokernel** is a kernel that **does not control how programs use hardware**.

Instead, it:

- Gives programs **direct access to hardware**
 - Only **protects** hardware so programs don't interfere with each other
-

What is a Process?

A **process** is a **program that is currently being executed**.

It includes the program code **and** its current activity.

Process State

The **process state** shows the **current condition of a process** at a particular time.

Common states:

New, Ready, Running, Waiting (Blocked), Terminated

Context Switching

Context switching is the process of **saving the state of a running process** and **loading the state of another process** so the CPU can switch between them.

Turnaround Time

Turnaround time is the **total time taken from process submission to its completion**.

Formula:

Turnaround Time = Completion Time – Arrival Time

Waiting Time

Waiting time is the **total time a process spends waiting in the ready queue** for CPU execution.

Formula:

Waiting Time = Turnaround Time – Burst Time

Response Time

Response time is the **time from process submission to the first time it gets CPU**.

Process Control Block (PCB)

A **Process Control Block (PCB)** is a data structure used by the operating system to store all important information about a process.

The OS uses PCB to **manage, schedule, and control** processes during execution.

Fields of PCB

1. Process ID (PID)

A unique identification number assigned to each process.

2. Process State

Shows the current status of the process.

Possible states:

- New
- Ready
- Running
- Waiting (Blocked)
- Terminated

3. Program Counter

Stores the address of the next instruction that the CPU will execute.

4. CPU Registers

Stores values of CPU registers such as:

- Accumulator
- Stack pointer
- Index registers

These values are saved during context switching.

5. Memory Information

Contains details about memory allocated to the process, such as:

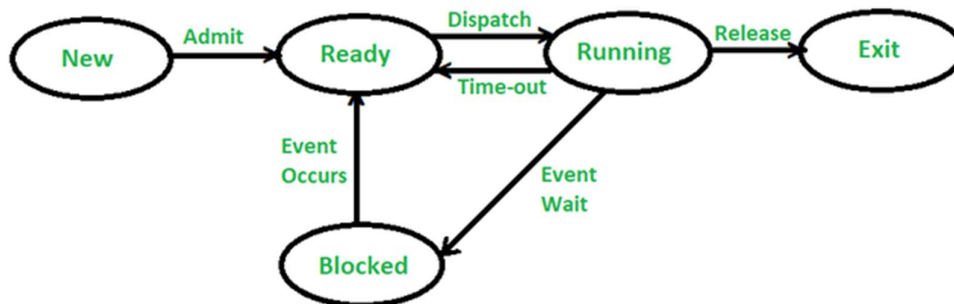
- Base address
- Limit register
- Page table or segment table

6. Scheduling Information

Contains data used by the CPU scheduler, such as:

- Process priority
- Scheduling queue
- CPU time used

Explain process states with state transition diagram.



Process States in an Operating System

A **process** is a program in execution.

During its lifetime, a process moves through **different states**.

1. New State

- The process is **created**.
- The operating system creates and initializes the **Process Control Block (PCB)**.
- The process is **not yet ready** to run.

2. Ready State

- The process is loaded into **main memory**.
- It is **waiting for CPU allocation**.
- The process can start execution as soon as the CPU becomes available.

3. Running State

- The process is **currently executing** on the CPU.
- At any time, **only one process can run on a single CPU core**.

4. Waiting (Blocked) State

- The process **cannot continue execution** until a specific event occurs.
- The CPU is **not required** during this state.

Examples of events

- File read completion
- Keyboard or user input

5. Terminated (Exit) State

- The process has **completed execution**.
- The operating system **releases all resources** used by the process.
- The PCB is removed from memory.

What is a Thread?

A thread is the smallest unit inside a process that can actually run on the CPU.

A process is made up of one or more threads, and each thread performs a specific job independently.

Threads inside one process **do not have their own separate memory**, so All threads in a process share the same memory and resources.

Why is a Thread Called “Lightweight”?

A thread is called lightweight because it requires fewer system resources than a process.

Reasons:

- It **does not have its own separate address space**.
- It **shares resources** like memory, files, and data with other threads of the same process.
- It uses **less memory**.
- It takes **less time to switch** between threads than between processes.

Advantages of Threads Over Processes

1. Faster Creation

- Creating a thread is quicker and simpler than creating a full process.

2. Lower Resource Usage

- Threads share memory and system resources with their parent process.
- This reduces overall system load.

3. Easy Communication

- Threads communicate directly through shared memory.
- Processes need special communication methods (IPC), which are slower and complex.

4. Parallel Execution

- Multiple threads can run at the same time on different CPU cores.
- This improves performance and speed.

Difference Between Process and Thread (Important)

Feature	Process	Thread
Definition	A program in execution	Smallest unit of execution inside a process
Nature	Heavyweight	Lightweight
Resource Sharing	Does not share resources by default	Shares resources with other threads
Creation Time	Takes more time	Takes less time
Context Switching	Slower	Faster
Failure Impact	One process crash usually does not affect others	One thread crash can affect the whole process

what is scheduling

Scheduling is the process by which the operating system decides the order in which processes run on the CPU to use it efficiently and fairly.

User-Level Threads (ULTs)

Definition:

Threads are managed by a user-level library, not by the operating system kernel.

Key Points:

- Created and controlled in **user space**.
- Kernel does **not know** about these threads.
- Thread operations are **fast** because no system call is needed.
- If one thread blocks, **all threads block** in that process.

Disadvantages:

- No true parallelism on multi-core CPUs.

Kernel-Level Threads (KLTs)

Definition:

Threads are managed directly by the operating system kernel.

Key Points:

- Kernel **knows and controls** each thread.
- Thread operations require **system calls**.
- If one thread blocks, **others can still run**.
- Supports **true parallel execution** on multi-core CPUs.

Disadvantages:

- Slower due to kernel involvement and system call.
-

Definition of Scheduler

A **scheduler** is a part of the operating system that **decides the order in which processes are executed by the CPU and other resources**.

It ensures efficient use of the CPU and proper system performance.

Types of Schedulers

1. Long-Term Scheduler (Job Scheduler)

- Decides **which jobs from the disk** will be loaded into memory.
- Controls **how many processes** run at the same time.
- Tries to keep a **good mix of CPU-bound and I/O-bound processes**.

2. Short-Term Scheduler (CPU Scheduler)

- Chooses the next process from the ready queue to use the CPU.
- Runs **very frequently** to keep the CPU busy.
- Ensures **fast and responsive execution**.

3. Medium-Term Scheduler

- Temporarily pauses processes in memory and resumes them later.
 - Helps balance CPU and I/O usage.
 - Improves overall system performance.
-

Non-Preemptive Scheduling

Non-preemptive scheduling is a CPU scheduling method where a process keeps the CPU **until it finishes** or moves to the waiting state. The operating system **cannot interrupt** it.

Types of Non-Preemptive Scheduling

1. FCFS (First Come First Serve)
 2. SJF (Shortest Job First – Non-Preemptive)
 3. Priority Scheduling (Non-Preemptive)
-

Preemptive Scheduling

Preemptive scheduling is a CPU scheduling method where the operating system **can interrupt a running process** and give the CPU to another process.

Types of Preemptive Scheduling

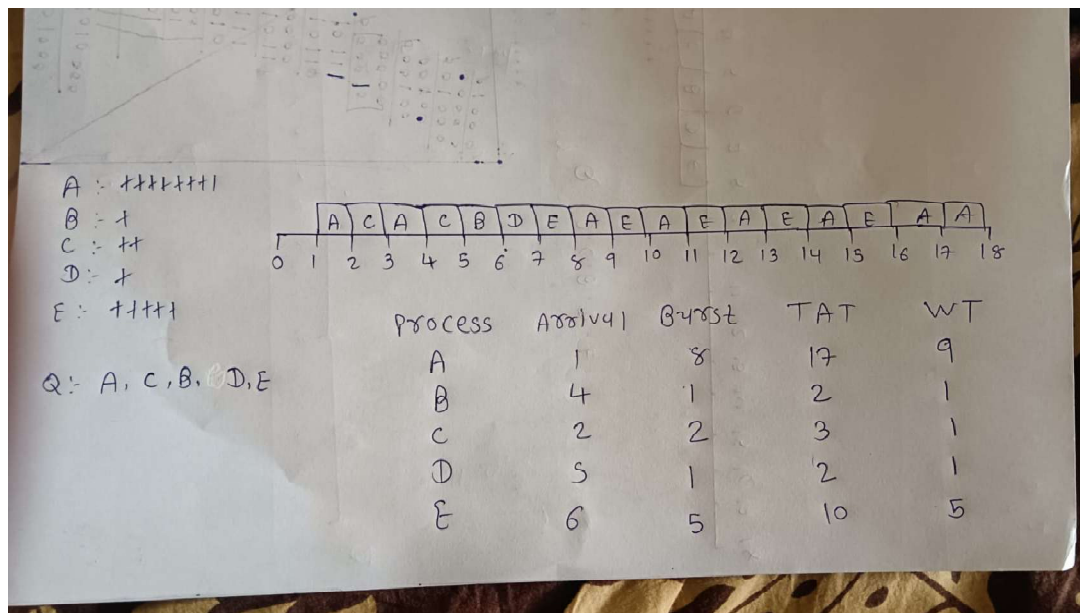
1. Round Robin (RR)
 2. SRTF (Shortest Remaining Time First)
 3. Priority Scheduling (Preemptive)
-

- (B) Consider five processes A, B, C, D and E with their arrival times and required CPU bursts as shown in the following table.

Process	A	B	C	D	E
Arrival Time	1	4	2	5	6
CPU bursts (ms)	8	1	2	1	5

Calculate their average turn around time and average waiting time using Round Robin Scheduling Algorithm. Provide the Gantt chart for the same. Time quantum is 1 ms.

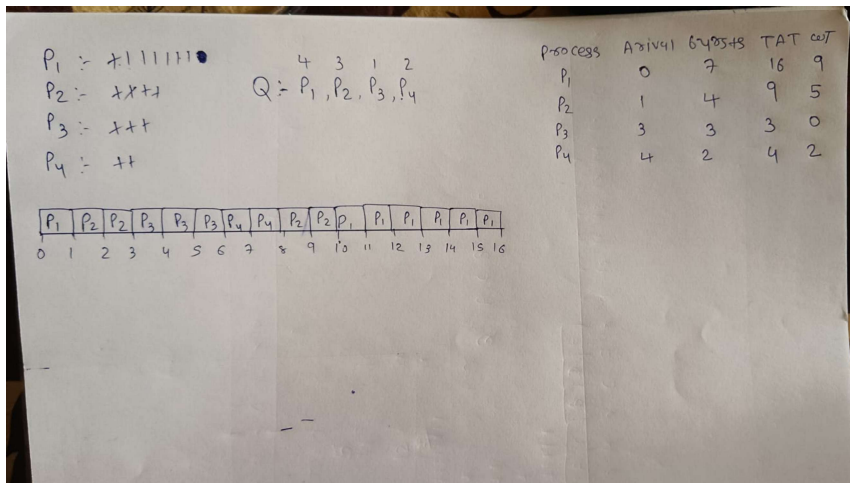
AD



- (B) Consider four processes P1, P2, P3 and P4 with their arrival times and required CPU bursts as shown in the following table.

Process	P1	P2	P3	P4
Arrival Time	0	1	3	4
CPU bursts (ms)	7	4	3	2
Priority	4	3	1	2

Calculate their average turn around time and average waiting time using Preemptive Priority Scheduling Algorithm. Provide the Gantt chart for the same. Assume lower priority number represents higher priority.



Q:2 [C] Define turnaround time and waiting time. Consider the system with following d

Process	Arrival Time	Execute Time	Priority
P0	0	5	1
P1	1	3	2
P2	2	8	1
P3	3	6	3

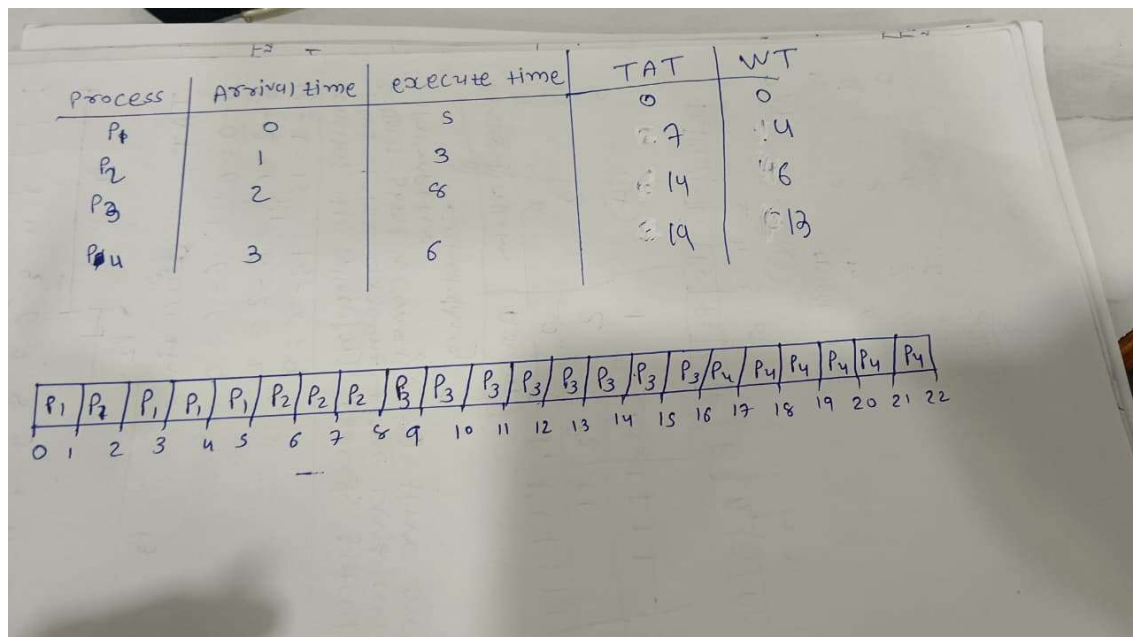
Find out average waiting time for the system using

a) First Come First Serve

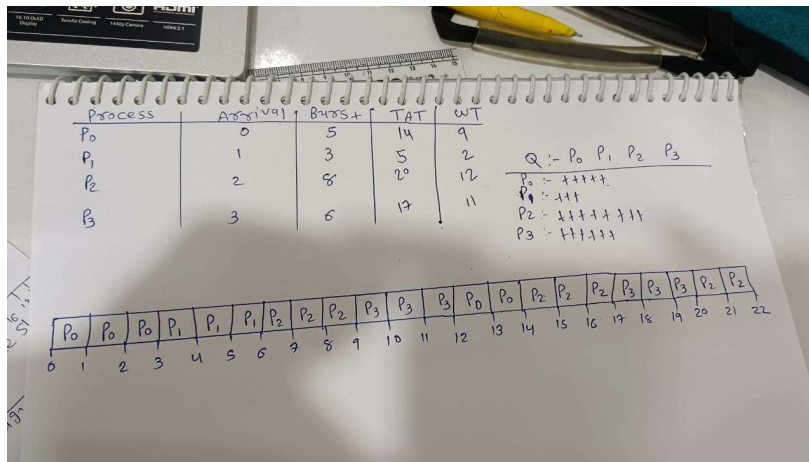
b) Round Robin Scheduling (quantum=3, ignore context switching overhead)

OR

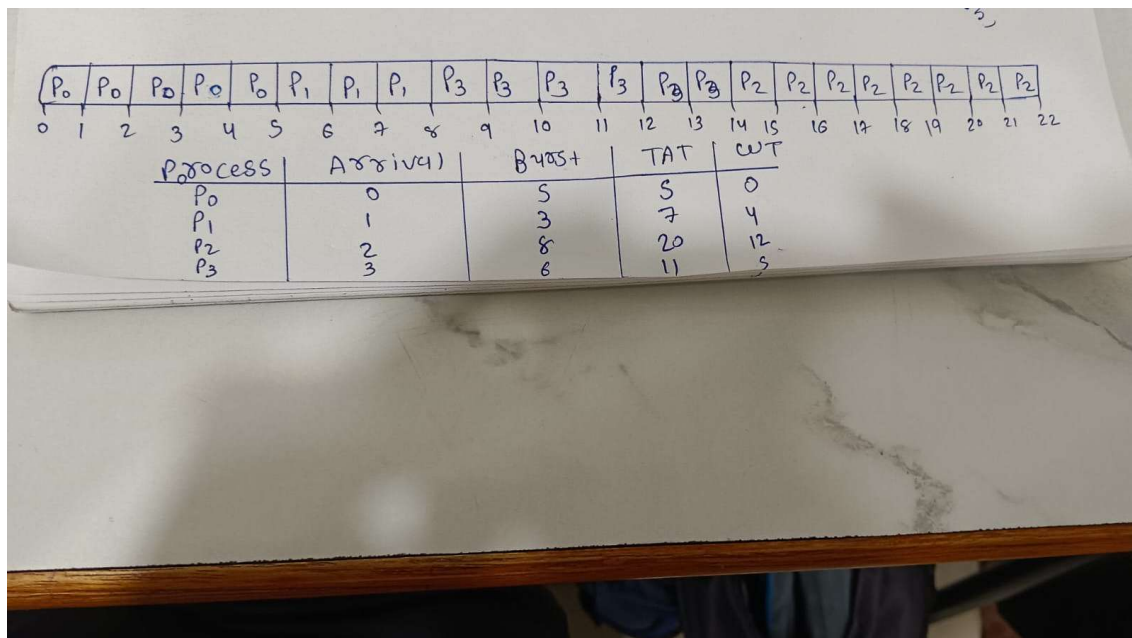
(a)



(b)



(c) Shortest Job First



CPU-Bound vs I/O-Bound Processes

CPU-Bound Process

- Uses CPU most of the time
- Performs heavy calculations
- Very little waiting for I/O

Example: Mathematical computations, video encoding.

I/O-Bound Process

- Spends most time waiting for input/output

- Uses CPU only for short time

Example: File reading, database queries.

Race Condition

A **race condition** occurs when two or more processes access shared data at the same time, and the final result depends on the order of execution.

This can cause incorrect results.

Critical Section

A **critical section** is a part of a program where a process accesses shared resources (memory, files, variables).

Only one process should execute it at a time to avoid errors.

Mutual Exclusion

Mutual exclusion is a rule in operating systems that ensures **only one process at a time** can enter a shared resource or critical section.

Spin Lock

A **spin lock** is a locking method where a process **keeps checking repeatedly** (busy waiting) until the lock becomes free.

File and Text Commands

1. cat

- Displays the contents of a file
- Creates new files
- Combines multiple files into one

2. grep

- Searches for specific text patterns inside files
- Supports regular expressions for advanced searching

3. ls

- Lists files and directories in a folder

4. head

- Displays the first 10 lines of a file by default
 - Number of lines can be changed using -n
-

Process Management Commands

5. ps

- Shows currently running processes
- Displays PID, status, and other details

6. kill

- Terminates a process using its Process ID (PID)
-

File Permission Command

7. chmod

- Changes file or directory permissions
 - Controls read, write, execute access
-

Utility Command

8. bc

- Command-line calculator
- Supports arithmetic and floating-point calculations

